Chapter 5 - Python Classes and Objects

October 15, 2020

---

1 Chapter 5 - Python Classes and Objects

A class is a user-defined blueprint or prototype from which objects are created. Classes provide a means of bundling data and functionality together. Creating a new class creates a new type of object, allowing new instances of that type to be made. Each class instance can have attributes attached to it for maintaining its state. Class instances can also have methods (defined by its class) for modifying its state.

To understand the need for creating a class let's consider an example, let's say you wanted to track the number of dogs which may have different attributes like breed, age. If a list is used, the first element could be the dog's breed while the second element could represent its age. Let's suppose there are 100 different dogs, then how would you know which element is supposed to be which? What if you wanted to add other properties to these dogs? This lacks organization and it's the exact need for classes.

Class creates a user-defined data structure, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. Some points on Python class:

```
Classes are created by keyword class.
Attributes are the variables that belong to class.
Attributes are always public and can be accessed using dot (.) operator.
```

So, Python is an object oriented programming language and almost everything in Python is an object, with its properties and methods. A Class is like an object constructor, or a "blueprint" for creating objects.

```
##Class Definition Syntax:

class ClassName:
    # Statement-1
    .
    .
    .
    # Statement-N
```

[1]:
```
# Python program to demonstrate defining a class
```

```
class Dog:
    pass # to avoid error
```

In the above example, class keyword indicates that you are creating a class followed by the name of the class

---

1.1 Class Objects

An Object is an instance of a Class. A class is like a blueprint while an instance is a copy of the class with actual values. It's not an idea anymore, it's an actual dog, like a dog of breed pug who's seven years old. You can have many dogs to create many different instances, but without the class as a guide, you would be lost, not knowing what information is required.

An object consists of:

```
State : It is represented by attributes of an object. It also reflects the properties of an obj
Behavior : It is represented by methods of an object. It also reflects the response of an objec
Identity : It gives a unique name to an object and enables one object to interact with other ob
```

[2]:
```
## Declaring Objects (Class) example

class Dog:

    # A simple class attribute
    attr1 = "hurtta"
    attr2 = "dog"

    # A sample method
    def fun(self):
        print("I'm a", self.attr1)
        print("I'm a", self.attr2)

# Driver code
# Object instantiation
Roger = Dog()

# Accessing class attributes and method through objects
print(Roger.attr1)
Roger.fun() # call method fun() which is member of Dog Class, so we use Roger␣
 ↪object to call method
```

```
hurtta
I'm a hurtta
I'm a dog
```

1.2 The self

Class methods must have an extra first parameter in method definition. We do not give a value for this parameter when we call the method, Python provides it. If we have a method which takes no arguments, then we still have to have one argument. This is similar to this pointer in C++ and this reference in Java.

1.3 Class and Instance Variables

Instance variables are for data unique to each instance and class variables are for attributes and methods shared by all instances of the class. Instance variables are variables whose value is assigned inside a constructor or method with self whereas class variables are variables whose value is assigned in the class.

Defining instance varibale using constructor.

1.3.1 init method

The `__init__` method is similar to constructors in C++ and Java. Constructors are used to initialize the object's state. Like methods, a constructor also contains a collection of statements(i.e. instructions) that are executed at the time of Object creation. It is run as soon as an object of a class is instantiated. The method is useful to do any initialization you want to do with your object.

[5]:
```
# Class for Dog
class Dog:

    # Class Variable
    animal = 'dog'

    # The init method or constructor
    def __init__(self, breed, color): ## constructor takes two variable

        # Instance Variable
        self.breed = breed
        self.color = color

# Objects of Dog class
Roger = Dog("Pug", "brown")  ## call constructor with two variables
Puksu = Dog("Bulldog", "black")

## Now we have two different dogs
print('Roger details:')
print('Roger is a', Roger.animal)
print('Breed: ', Roger.breed)
print('Color: ', Roger.color)
```

```
print('\nPuksu details:')
print('Puksu is a', Puksu.animal)
print('Breed: ', Puksu.breed)
print('Color: ', Puksu.color)

# Class variables can be accessed using class name also
print("\nAccessing class variable using class name")
print(Dog.animal)  ## This print class variables
```

```
Roger details:
Roger is a dog
Breed:  Pug
Color:  brown

Puksu details:
Puksu is a dog
Breed:  Bulldog
Color:  black

Accessing class variable using class name
dog
```

[6]:
```
## Some more class examples

# Create a class named Person, use the __init__() function to assign values for␣
 ↪name and age:
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("Eero", 50)

print(p1.name)
print(p1.age)
```

```
Eero
50
```

Note: The init() function is called automatically every time the class is being used to create a new object.

1.4 Object Methods

Objects can also contain methods. Methods in objects are functions that belong to the object.

Let us create a method in the Person class:

[11]:
```
# Insert a function that prints a greeting, and execute it on the p1 object:
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name + " and I'm " + str(self.age) + " old␣
 ↪gubbe")

p1 = Person("Eero", 50)
p1.myfunc()
```

```
Hello my name is Eero and I'm 50 old gubbe
```

Note: The self parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

1.5 The self Parameter

The self parameter is a reference to the current instance of the class as we know, and is used to access variables that belongs to the class.

It does not have to be named self , you can call it whatever you like, but it has to be the first parameter of any function in the class:

[12]:
```
# Use the words mysillyobject and abc instead of self:
class Person:
  def __init__(mysillyobject, name, age):
    mysillyobject.name = name
    mysillyobject.age = age

  def myfunc(abc):
    print("Hello my name is " + abc.name)

p1 = Person("Eero", 50)
p1.myfunc()
```

```
Hello my name is Eero
```

1.6 Modify Object Properties

You can modify properties on objects like this:

[22]:
```
class Person:
  def __init__(mysillyobject, name, age):
    mysillyobject.name = name
    mysillyobject.age = age

  def myfunc(abc):
    print("Hello my name is " + abc.name + " and " + str(abc.age) + " old␣
 ↪gubbe")

p1 = Person("Eero", 50)
p1.myfunc()

print("Hah...now so young....so have to set the new age of p1 object 60")
p1.age = 60

p1.myfunc()
```

```
Hello my name is Eero and 50 old gubbe
Hah…now so young…so have to set the new age of p1 object 60
Hello my name is Eero and 60 old gubbe
```

1.7 Delete Object Properties

You can delete properties on objects by using the del keyword:

[25]:
```
class Person:
  def __init__(mysillyobject, name, age):
    mysillyobject.name = name
    mysillyobject.age = age

  def myfunc(abc):
    print("Hello my name is " + abc.name + " and " + str(abc.age) + " old␣
 ↪gubbe")

  def myage(abc):
    print("Hello my name is " + abc.name)

p1 = Person("Eero", 50)
p1.myfunc()

print("Hah...now so young....so have to set the new age of p1 object 60")
p1.age = 60

p1.myfunc()
## delete age properties
```

```
del p1.age
## print new one person object calling myage
p1.myage()
```

```
Hello my name is Eero and 50 old gubbe
Hah…now so young…so have to set the new age of p1 object 60
Hello my name is Eero and 60 old gubbe
Hello my name is Eero
```

1.8 Delete Objects

You can delete objects by using the del keyword:

[35]:
```
class Person:
  def __init__(mysillyobject, name, age):
    mysillyobject.name = name
    mysillyobject.age = age

  def myfunc(abc):
    print("Hello my name is " + abc.name + " and " + str(abc.age) + " old␣
 ↪gubbe")

  def myage(abc):
    print("Hello my name is " + abc.name)

p1 = Person("Eero", 50)
p1.myfunc()

print("Hah...now so young....so have to set the new age of p1 object 60")
p1.age = 60

p1.myfunc()
## delete age properties
del p1.age
## print new one person object calling myage
p1.myage()

del p1

#p1.myage ## these method call causes NameError

#This is a several lines comment
""" NameError Traceback (most recent call last)
<ipython-input-29-5d93b7f7fc04> in <module>
     24 del p1
```

```
    25
---> 26 p1.myage ## these method call causes NameError

NameError: name 'p1' is not defined"""
```

```
Hello my name is Eero and 50 old gubbe
Hah…now so young…so have to set the new age of p1 object 60
Hello my name is Eero and 60 old gubbe
Hello my name is Eero
```

[35]:
```
'" NameError Traceback (most recent call last)\n<ipython-input-29-5d93b7f7fc04>
in <module>\n     24 del p1\n     25 \n---> 26 p1.myage ## these method call
causes NameError\n\nNameError: name \'p1\' is not defined'
```

---

2 Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from base class.

Child class is the class that inherits from another derived class.

[36]:
```
#Create a class named Person, with firstname and lastname properties, and a␣
↪printname method:
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

#Use the Person class to create an object, and then execute the printname␣
↪method:

x = Person("Eero", "Huusko")
x.printname()
```

```
Eero Huusko
```

[38]:
```
# Create a class named Lecturer, which will inherit the properties and methods␣
↪from the Person class:
class Lecturer(Person):
  pass # to avoid error
```

[39]:
```
#Use the Lecturer class to create an object, and then execute the printname␣
↪method:
x = Lecturer("Eero", "Huusko")
x.printname()
```

```
Eero Huusko
```

2.1 Add the init() Function

So far we have created a child class that inherits the properties and methods from its parent.

We want to add the `__init__()` function to the child class instead of the pass keyword. When adding the `__init__()` function, the child class will no longer inherit the parent's `__init__()` function.

[ ]:
```
#Add the __init__() function to the Lecturer class:
class Lecturer(Person):
  def __init__(self, fname, lname):

#add properties etc.
```

To keep the inheritance of the parent's `__init__()` function, add a call to the parent's `__init__()` function:

[44]:
```
class Lecturer(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
```

2.2 Use the super() Function

Like Java, Python also has a super() function that will make the child class inherit all the methods and properties from its parent. By using the super() function, you do not have to use the name of the parent element, it will automatically inherit the methods and properties from its parent.

[46]:
```
class Lecturer(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
```

2.3 Add Properties

Inherit Student class from Person class and add a property called graduationyear to the Student class:

[48]:
```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
    self.graduationyear = 2020
```

In the example below, the year 2020 should be a variable, and passed into the Student class when creating student objects. To do so, add another parameter in the **init**() function:

2.4 Creating student object

Add a year parameter, and pass the correct year when creating objects:

[49]:
```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

studentObject = Student("Mikko", "Kettunen", 2020)
```

2.5 Add Methods

If you add a method in the child class with the same name as a function in the parent class, the inheritance of the parent method will be overridden.

[52]:
```
#Add a method called welcome to the Student class:
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the class of", self.
 ↪graduationyear)

studentObject = Student("Mikko", "Kettunen", 2020)

studentObject.welcome()
studentObject.printname()# now we call parent class method printname()
```

```
Welcome Mikko Kettunen to the class of 2020
Mikko Kettunen
```

---

3 Python Try Except

The try block lets you test a block of code for errors.

The except block lets you handle the error.

The finally block lets you execute code, regardless of the result of the try- and except blocks.

3.1 Exception Handling

When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

These exceptions can be handled using the try statement:

[ ]:
```
try:
  print(x)
except NameError:
  print("Variable x is not defined")
except:
  print("Something else went wrong")
```

[70]:
```
## Example - Try to open and write to a file that is not writable:
try:
  f = open("demofile.txt")
  f.write("Lorum Ipsum")
except:
  print("Something went wrong when writing to the file")
finally:
  f.close()
```

```
Something went wrong when writing to the file
```

```
      ␣
↪---------------------------------------------------------------------------

       NameError                                 Traceback (most recent call␣
↪last)

       <ipython-input-70-2810441e0195> in <module>
         6   print("Something went wrong when writing to the file")
         7 finally:
   ----> 8   f.close()
```

```
NameError: name 'f' is not defined
```

4 So, that the end of this story – Let's go to the challenges and make some exercises

[ ]: